

Le VPC : votre réseau dans le cloud

Le plan que vous avez dessiné sur papier, aujourd'hui on le construit

CL-AWS1 — Jour 3 — Construire, sécuriser, puis scripter votre réseau AWS

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- **Construire à la console** un VPC complet : 4 sous-réseaux sur 2 AZ, Internet Gateway, tables de routage, NAT Gateway — composant par composant, dans le bon ordre.
- **Expliquer** ce qui rend un sous-réseau « public » (indice : ce n'est pas son nom).
- **Configurer des security groups** en profondeur : stateful, allow only, source = un autre SG.
- **Décider** quand toucher aux NACL — et quand les laisser tranquilles.
- **Atteindre une instance privée** via un bastion (`ssh -J`, votre réflexe de CL-LINUX) et **prouver** qu'elle sort par la NAT Gateway sans être joignable de l'extérieur.
- **Rejouer toute la construction en script CLI** — et tout détruire proprement chaque soir.

Fil conducteur, toujours : **automatiser, superviser, sécuriser**. Ce qui prend 40 minutes à la console prendra 4 minutes en script. C'est votre métier.

Plan de la journée

Matin

1. **Rappel-pont** — ce que vous savez déjà du bloc réseau : aujourd'hui on clique, puis on scripte.
2. **Le VPC, composant par composant** — VPC, sous-réseaux, IGW, tables de routage, NAT Gateway 💰.
💻 Démo 3.1 (temps 1) : construction à la console + bastion + preuve de la sortie NAT.
3. **Security groups en profondeur** — stateful, allow only, référence de SG à SG.

Après-midi

4. 🛠 Exercice 3.1 — un VPC pour le client « Datalis » (cahier des charges différent de la démo).
5. **NACL** — stateless, règles numérotées, ports éphémères. 🛠 Exercice 3.2 (sur papier).
6. « **Tout est API** » — 💻 Démo 3.1 (temps 2) : `create-vpc.sh` et `teardown-vpc.sh`.
7. 🧑🏻 **Mini-TP** — reconstruire tout le VPC en autonomie. Non noté, mais c'est votre répétition générale.

Puis : récap, quiz de fin de journée, et le teaser du Jour 4.

1. Rappel-pont

Du papier au cloud : vous savez déjà presque tout

Ce que vous savez déjà (bloc CL-RÉSEAU)

Rien de ce matin n'est nouveau **conceptuellement**. Vous avez déjà :

- **Découpé des CIDR** : $10.0.0.0/16 = 65\,536$ adresses, un $/24 = 256$ adresses.
- **Compris le routage** : une table, des destinations, des prochains sauts, la route la plus spécifique gagne.
- **Vu le NAT** : plusieurs machines privées sortent sur Internet via une seule IP publique — le retour est traduit, mais personne ne peut *initier* une connexion vers elles.
- **Opposé stateful et stateless** : un pare-feu stateful se souvient des connexions (le retour passe tout seul) ; un stateless examine chaque paquet isolément.
- **Dessiné le plan d'adressage StockLine** : VPC, sous-réseaux publics/privés sur 2 AZ, tables de routage, trois SG avec références croisées.

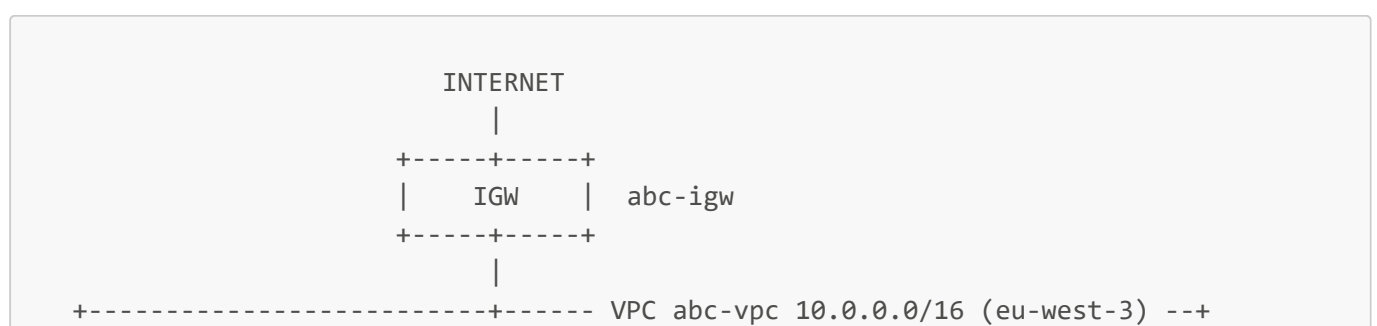
Ce qui change aujourd'hui : ces concepts deviennent des **ressources AWS avec des identifiants** (`vpc-0abc...`, `subnet-0def...`, `igw-...`, `nat-...`) que l'on crée, relie et détruit — à la souris ce matin, **en script ce soir**.

Le dictionnaire : votre schéma papier → les objets AWS

Sur votre plan papier (CL-RÉSEAU J4)	Chez AWS	Identifiant
« Notre réseau » $10.x.0.0/16$	VPC	<code>vpc-...</code>
Un sous-réseau dans une salle machine	Subnet — vit dans une seule AZ	<code>subnet-...</code>
La porte vers Internet	Internet Gateway (IGW)	<code>igw-...</code>
La table de routage du routeur	Route table — associée à des sous-réseaux	<code>rtb-...</code>
Le boîtier NAT de sortie	NAT Gateway 💰 + Elastic IP	<code>nat-...</code> + <code>eipalloc-...</code>
Le pare-feu stateful par machine	Security group	<code>sg-...</code>
Le filtre stateless par sous-réseau	Network ACL (NACL)	<code>acl-...</code>

Un VPC est **régional** (le nôtre : eu-west-3, Paris) et s'étend sur toutes les AZ de la région. Un sous-réseau, lui, est **zonal** : c'est le grain de la haute disponibilité.

L'architecture cible du jour (gardez-la sous les yeux)



```

| table PUBLIQUE : 10.0.0.0/16 -> local | 0.0.0.0/0 -> IGW |
| +-----+ +-----+ |
| | abc-public-a (3a) | | abc-public-b (3b) | |
| | 10.0.0.0/24 | | 10.0.1.0/24 | |
| | [bastion] [NAT GW 💰] | | | |
| +-----+ +-----+ |
| table PRIVÉE : 10.0.0.0/16 -> local | 0.0.0.0/0 -> NAT GW |
| +-----+ +-----+ |
| | abc-private-a (3a) | | abc-private-b (3b) | |
| | 10.0.10.0/24 | | 10.0.11.0/24 | |
| | [instance privée] | | | |
| +-----+ +-----+ |
+-----+
SG abc-sg-bastion : 22/tcp depuis VOTRE_IP/32
SG abc-sg-private : 22/tcp depuis abc-sg-bastion (référence SG -> SG)

```

2. Le VPC, composant par composant

On assemble à la main — pour comprendre chaque pièce

Le VPC : votre morceau de réseau, à vous

Un **VPC** (Virtual Private Cloud), c'est un bloc d'adresses privées (RFC 1918) réservé pour vous dans une région :

- Le nôtre : **10.0.0.0/16** → **65 536 adresses**, dans eu-west-3.
- Personne d'autre n'y entre ; rien n'en sort **tant qu'on ne l'a pas décidé**. Un VPC neuf est une boîte hermétique : pas d'Internet, ni en entrée ni en sortie.
- Le choix du CIDR est **définitif** (on ne renumérote pas, on ajoute ou on détruit) — d'où l'atelier papier du bloc réseau : pas de chevauchement avec un futur VPC dev, de la place pour grandir.

À la console : **VPC** → **Your VPCs** → **Create VPC**, option « **VPC only** ». L'assistant « VPC and more » ferait tout à notre place — précisément ce qu'on ne veut pas aujourd'hui : **on apprend en assemblant**.

Deux réglages à activer sur le VPC : **DNS resolution** et **DNS hostnames** (indispensables plus tard pour RDS).

Les sous-réseaux : un sous-réseau = une AZ

On découpe le **/16** en sous-réseaux. Règle absolue : **un sous-réseau vit dans une seule zone de disponibilité**. C'est le grain de la haute dispo : pour survivre à la panne d'une AZ, il faut des sous-réseaux dans au moins deux AZ.

Nom	CIDR	AZ	Rôle
abc-public-a	10.0.0.0/24	eu-west-3a	bastion, NAT GW
abc-public-b	10.0.1.0/24	eu-west-3b	(réserve publique)

Nom	CIDR	AZ	Rôle
abc-private-a	10.0.10.0/24	eu-west-3a	instances applicatives
abc-private-b	10.0.11.0/24	eu-west-3b	instances applicatives

Remarquez les **trous volontaires** (10.0.2 à 10.0.9) : la place pour grandir sans renuméroter — exactement la « question qui fâche » de l'atelier papier.

À ce stade, « public » et « privé » ne sont **que des noms**. Rien ne les distingue encore.

Les 5 adresses que AWS vous confisque

Dans **chaque** sous-réseau, AWS réserve 5 adresses. Pour **10.0.0.0/24** :

Adresse	Usage
10.0.0.0	Adresse de réseau (comme en réseau classique)
10.0.0.1	Le routeur du VPC (la passerelle implicite de vos instances)
10.0.0.2	Le résolveur DNS d'AWS (l'adresse « base du réseau + 2 »)
10.0.0.3	Réservée par AWS pour usage futur
10.0.0.255	Adresse de broadcast (le broadcast n'existe pas dans un VPC, mais elle reste réservée)

Donc un **/24 AWS = 251 adresses utilisables**, pas 254 comme en réseau classique. Un **/20 = 4 096 – 5 = 4 091 utilisables**. (Retenez « – 5 », ça revient au quiz... et en entretien.)

Plus petit sous-réseau autorisé : **/28** (16 – 5 = 11 utilisables). Plus grand : **/16**.

L'Internet Gateway : créer, PUIS attacher

L'**IGW** est la porte entre le VPC et Internet. Sans elle, aucun paquet n'entre ni ne sort, quoi qu'on route.

Particularité qui surprend : c'est **deux opérations distinctes** —

1. **Créer** l'IGW → elle naît à l'état **detached**, rattachée à rien.
2. **L'attacher** au VPC → une IGW ne sert qu'un seul VPC à la fois.

```
aws ec2 create-internet-gateway ... # naissance, état "detached"
aws ec2 attach-internet-gateway --internet-gateway-id igw-... --vpc-id vpc-...
```

Vous retrouverez ce motif partout chez AWS : **créer une ressource** et **la relier** sont deux actes séparés (IGW/VPC, EIP/NAT, table de routage/sous-réseau...). Le teardown devra défaire les deux, dans l'ordre inverse : **détacher puis supprimer**.

Et attention : attacher une IGW ne rend **rien** public. Il manque encore la route.

Les tables de routage — et LA règle de la journée

Chaque VPC naît avec une table *main* contenant une route **insupprimable** : `10.0.0.0/16 → local` (tout le VPC se parle en interne). Bonne pratique : ne pas toucher à la *main* — créer des tables dédiées et les **associer** : `abc-rt-public → public-a/b`, `abc-rt-private → private-a/b`.

Un sous-réseau est **public** si — et seulement si — sa table de routage contient :

Destination	Cible
<code>0.0.0.0/0</code>	<code>igw-...</code>

Pas son nom. Pas une case à cocher « public ». Cette route, et rien d'autre. Renommez `abc-private-a` en `super-public` : il reste privé.

Deuxième ingrédient : l'attribut **auto-assign public IPv4** du sous-réseau (sans lui, pas d'IP publique au lancement). Pour qu'une instance soit joignable depuis Internet, il faut donc **les trois** : route `0.0.0.0/0 → IGW` + IP publique + SG qui laisse entrer.

La NAT Gateway : sortir sans être joignable

À ce stade, nos sous-réseaux privés (restés sur la table *main*) se parlent en interne... mais ne peuvent même pas faire un `apt update`. Il leur faut une **sortie sans entrée** : la **NAT Gateway**, le NAT managé d'AWS — le concept vu au bloc réseau, en service clé en main :

- Les instances **privées** sortent (dépôts apt, API externes) **via l'IP de la NAT GW** ;
- personne ne peut **initier** une connexion entrante vers elles ;
- managée : pas de machine à administrer, tient jusqu'à 100 Gb/s, réparée par AWS.

Trois règles de placement à retenir (sources classiques d'erreur) :

1. Elle vit dans un sous-réseau **PUBLIC** — c'est elle qui a besoin d'aller sur Internet ! Une NAT GW dans un sous-réseau privé se crée sans erreur... et ne fonctionne jamais.
2. Elle a besoin d'une **Elastic IP** (EIP) : une IP publique fixe, allouée **avant**, associée à la création.
3. Elle est **zonale** : la nôtre en `public-a`. En production critique, on en met une **par AZ** (arbitrage coût/panne — votre débat de l'atelier papier).

Elle met **1 à 3 minutes** à passer de `pending` à `available`. On patiente avant de router.

💰 La NAT Gateway est FACTURÉE — le rituel du soir

💰 Coût — à graver dans votre routine

- NAT Gateway : $\approx 0,05 / \text{heure} * * (\text{même inutilisée!}) * * + \approx 0,05 / \text{Go}$ traité.
- Soit $\approx 1,20 / \text{jour} * *, ** \approx 36 / \text{mois}$ si vous l'oubliez. Votre alerte budget du J1 (10 \$) hurlerait en une semaine.
- L'**Elastic IP** est gratuite tant qu'elle est associée à une ressource qui tourne, mais **facturée si elle reste allouée dans le vide** — une EIP orpheline après un teardown bâclé, c'est la fuite classique.

Le rituel du soir, non négociable :

```
./teardown-vpc.sh      # chaque soir, avant de fermer le laptop
```

La NAT Gateway **ne survit jamais à la journée**. C'est la première ressource vraiment payante du cursus : c'est aujourd'hui qu'on prend le pli. *Automatiser, superviser... et éteindre.*

La table privée : la boucle est bouclée

Dernière pièce du routage : **abc-rt-private**, associée aux deux sous-réseaux privés :

Destination	Cible	Effet
10.0.0.0/16	local	parler à tout le VPC (bastion compris)
0.0.0.0/0	nat-...	sortir sur Internet via la NAT GW

Le schéma de routage, complet cette fois :

```

      INTERNET
      |
      [ IGW ] <--- rt-public : 0.0.0.0/0 -> IGW
              | (entrée ET sortie possibles)
[ NAT GW 💰 en public-a ] <--- rt-private : 0.0.0.0/0 -> NAT GW
      ^
[ instances privées ]  sortie : OUI, via l'EIP de la NAT
                       entrée initiée d'Internet : JAMAIS

```

Vu de l'extérieur, tout le trafic sortant des instances privées porte **l'EIP de la NAT GW** — c'est la preuve que nous ferons tout à l'heure avec un simple **curl**.

Le squelette réseau est complet. Reste à poser les pare-feu : **security groups**.

3. Security groups en profondeur

Le pare-feu stateful qui entoure chaque instance

Le SG : un videur autour de l'instance

Un **security group** est un pare-feu virtuel attaché à l'**instance** (précisément : à sa carte réseau, l'ENI) — pas au sous-réseau.

- Règles **entrantes** (inbound) : qui a le droit d'initier une connexion **vers** l'instance.
- Règles **sortantes** (outbound) : ce que l'instance a le droit d'initier vers l'extérieur (par défaut : tout).

- Une règle = protocole + port(s) + **source** (en entrée) ou destination (en sortie).
- Une instance peut porter plusieurs SG ; un SG peut servir à des dizaines d'instances.

Le **SG par défaut** de chaque VPC : les membres du groupe se parlent entre eux, tout le reste est bloqué en entrée. Pratique, mais opaque — en formation comme en prod, on crée des SG **nommés et explicites** (`abc-sg-bastion`, `abc-sg-private`).

Vous en avez déjà utilisé un au J2 sans le disséquer. Aujourd'hui, on ouvre le capot.

Stateful : l'aller ouvre la porte du retour

Un SG est **stateful** : il mémorise les connexions. Si l'aller est autorisé, **le retour passe automatiquement** — quelles que soient les règles sortantes.

Votre poste	Instance (SG: IN 22/tcp depuis vous)
--- SYN vers :22 ----->	IN autorisé -> PASSE
	[le SG note la connexion]
<-- SYN-ACK depuis :22 -----	retour d'une connexion
	connue -> PASSE, sans
	consulter les règles OUT

Conséquences pratiques :

- Pour du SSH entrant, **une seule règle suffit** : `IN 22/tcp` depuis votre IP.
- Videz toutes les règles sortantes : le serveur **répond toujours** aux connexions entrantes... mais ne peut plus **initier** quoi que ce soit (adieu `apt update`).

Gardez ce schéma en tête : la NACL, tout à l'heure, fera **exactement l'inverse**.

Allow only : ce qu'un SG ne sait pas faire

Un security group ne contient **que des autorisations** :

- **Pas de règle « deny »** — impossible d'écrire « tout le monde sauf cette IP ».
- Tout ce qui ne correspond à aucune règle est **refusé implicitement** (même logique « défaut fermé » que le deny implicite d'IAM au J1 — AWS est cohérent).
- Pas de numéros, pas d'ordre : **toutes les règles sont évaluées**, il suffit qu'une autorise.
- ICMP (le `ping`) est un protocole **à part** : autoriser TCP 22 et 80 n'autorise pas le ping. Un ping qui échoue vers une instance ne prouve donc pas qu'elle est en panne !

Besoin de **bannir** une IP précise qui vous scanne ? Le SG ne sait pas faire. Il faudra la NACL — section suivante.

Bonnes pratiques : jamais de `0.0.0.0/0` sur le port 22 (votre IP `/32` uniquement) ; un SG par rôle (`bastion`, `app`, `db`), pas un SG fourre-tout.

La source peut être... un autre SG (le patron du 3-tiers)

En entrée, la source n'est pas forcément un CIDR : ce peut être **l'identifiant d'un autre SG**. La règle suit alors les instances, **quelles que soient leurs IP**.

```

abc-sg-bastion                                abc-sg-private
IN: 22/tcp <- VOTRE_IP/32                     IN: 22/tcp <- sg(abc-sg-bastion)
      |                                         ^
      |                                         |
      |----- SSH -----+
      |
      | "toute instance PORTANT le SG bastion
      | peut me parler sur 22 – peu importe son IP"

```

C'est **le** patron d'architecture 3-tiers que vous avez conçu sur papier :

```

sg-alb : IN 443 <- 0.0.0.0/0
sg-app : IN 8000 <- sg-alb           (seul l'ALB parle à l'app)
sg-db  : IN 5432 <- sg-app           (seule l'app parle à la base)

```

Une instance app de plus ? **Aucune règle à modifier**. Les IP changent, les rôles restent.



Exercice 3.1 — Un VPC pour le client « Datalis »

60 min, seul ou en binôme — console OU CLI, au choix.

Un client fictif, **Datalis**, vous confie un cahier des charges **différent** de la démo :

- VPC **10.20.0.0/16**, trois sous-réseaux (un public **/24**, deux privés **/20** — app et db),
- **pas de NAT Gateway** (économie assumée — vous analyserez ce que ça implique),
- SG app (80/443 monde + SSH votre IP), SG db (5432 **depuis le SG app uniquement**).

Des questions de conception sont intercalées : combien d'IP utilisables dans un **/20** AWS ? Pourquoi la base n'a-t-elle aucune route vers Internet ?



Énoncé : [exercices/06-cl-aws1/exercice-3-1-vpc-cahier-des-charges.md](#)



Teardown en fin d'exercice (checklist incluse) — même sans NAT GW, on ne laisse rien traîner. Bonus : tout refaire en script CLI.

4. Les NACL

Le pare-feu stateless du sous-réseau — puissant, et piégeux

NACL : l'autre pare-feu, à la frontière du sous-réseau

La **Network ACL** filtre le trafic **à l'entrée et à la sortie du sous-réseau** (pas de l'instance). Trois différences fondamentales avec le SG :

1. **Stateless** : aucune mémoire des connexions. Chaque paquet est jugé isolément — l'aller **et** le retour doivent être explicitement autorisés.
2. **Règles numérotées** : évaluées de la plus petite à la plus grande ; **la première qui correspond gagne** et l'évaluation s'arrête. La dernière est toujours * : deny.
3. **Allow ET deny** : la NACL sait **refuser** explicitement — c'est le seul des deux mécanismes qui sache bannir une IP.

La **NACL par défaut** de votre VPC : règle 100 = tout autoriser (entrée et sortie). Autrement dit : **transparente**. Chaque sous-réseau a exactement une NACL.

Périmètre : la NACL ne voit que le trafic qui **franchit la frontière du sous-réseau**. Deux instances du même sous-réseau ? La NACL ne s'applique pas — seuls leurs SG décident.

Le piège n° 1 : les ports éphémères au retour

Stateless = penser au **retour**. Or votre client SSH ne part pas du port 22 : il part d'un **port éphémère** aléatoire (Linux : 32768-60999 ; règle AWS usuelle : **1024-65535**).

Poste (port 54321)	Sous-réseau [NACL] Instance : 22
--- SYN 54321 -> 22 ----->	IN règle 100 allow 22 : PASSE
<-- SYN-ACK 22 -> 54321 -----	OUT ... qui autorise le
	port 54321 en sortie ?
X BLOQUÉ X	PERSONNE -> deny * -> DROP

Symptôme vécu : « le SG est bon, pourtant ça **timeout** »... parce qu'une NACL sortante ne rend pas le retour.

Règle d'or : toute NACL personnalisée qui autorise un service en entrée doit autoriser **TCP 1024-65535 en sortie** (et réciproquement pour le trafic initié depuis le sous-réseau).

Quand utiliser une NACL — et quand la laisser tranquille

Les bons cas d'usage (rares mais réels) :

- **Bannir une IP ou une plage** : **deny 203.0.113.66/32** avant la règle allow — le SG ne sait pas faire.
- **Défense en profondeur** : une deuxième ligne indépendante du SG — si quelqu'un ouvre un SG trop large par erreur, la NACL tient encore.
- **Exigence de conformité** : « le sous-réseau base de données ne doit accepter que le 5432 depuis le tier app » — écrit au niveau réseau, auditable.

Le reste du temps : la NACL par défaut (100 : allow all) fait très bien son travail — **ne la touchez pas**. Une NACL modifiée à la va-vite est la cause n° 1 des « pannes réseau fantômes » où tous les SG sont pourtant corrects.

Attention à l'ordre : **100 deny + 200 allow** $\neq$ **100 allow + 200 deny**. **Le numéro le plus bas qui correspond gagne — l'autre n'est jamais lu.**

SG vs NACL : le tableau à connaître par cœur

	Security group	NACL
S'applique à	l' instance (son ENI)	la frontière du sous-réseau
État	stateful — le retour passe seul	stateless — penser aux éphémères 1024-65535
Règles	allow uniquement (deny implicite)	allow ET deny explicites
Évaluation	toutes les règles, une suffit	par numéro croissant , la 1 ^{re} qui correspond gagne
Source possible	CIDR ou un autre SG	CIDR uniquement
Par défaut	tout refusé en entrée, tout autorisé en sortie	NACL par défaut : tout autorisé (transparente)
Bannir une IP	✗ impossible	✓ règle deny
Usage quotidien	votre outil principal	défense en profondeur, cas particuliers

Réflexe de dépannage « ça ne répond pas », dans l'ordre : **SG** → **table de routage** → **IP publique** → **NACL** (la NACL en dernier... sauf si quelqu'un y a touché).

Exercice 3.2 — SG vs NACL : qui bloque quoi ?

30-40 min, sur papier, seul puis correction collective.

Six scénarios réels : pour chacun, les tables SG et NACL vous sont fournies, et vous devez prédire — **paquet par paquet** — si la connexion aboutit, et sinon, **qui** bloque :

- le SSH qui timeout alors que « le SG est bon » ;
- le ping qui échoue vers une instance en parfaite santé ;
- l'IP malveillante à bannir sans rien casser ;
- **100 deny** contre **200 allow** — l'ordre qui change tout ;
- le SG sortant vidé... et le serveur qui répond quand même ;
- la NACL « béton » que deux instances contournent sans le savoir.

 Énoncé : [exercices/06-cl-aws1/exercice-3-2-sg-vs-nacl.md](#)

C'est un exercice d'**entretien d'embauche** autant que de certification. Aucun compte AWS requis : juste votre tête et les deux modèles mentaux (stateful / stateless).

5. Le bastion

Atteindre des instances qui n'ont pas d'adresse publique

Le bastion : un seul point d'entrée, jamais de clé dessus

Nos instances privées n'ont **pas d'IP publique** — voulu : surface d'attaque minimale. Pour les administrer : le **bastion** (jump host), petite instance en sous-réseau **public**, seul point d'entrée SSH, verrouillé sur **votre** IP :

```
Vous ---22---> [bastion 10.0.0.x + IP publique] ---22---> [privée 10.0.10.x]
(sg-bastion: 22 <- VOTRE_IP/32) (sg-private: 22 <- sg-bastion)
```

Le monde ne voit que le bastion ; le bastion ne parle qu'aux instances qui l'acceptent **par référence de SG**. Et la clé privée **ne quitte jamais votre poste** — on ne copie jamais `abc-key.pem` sur le bastion. L'outil : le **ProxyJump de CL-LINUX J4** + l'agent SSH :

```
ssh-add ~/.ssh/abc-key.pem # la clé entre dans l'agent
ssh -J ubuntu@IP_PUBLIQUE_BASTION ubuntu@10.0.10.54
```

L'agent authentifie **les deux sauts** sans que le bastion ne voie la clé. Variante : l'*agent forwarding* — `ssh -A ubuntu@IP_BASTION`, puis `ssh ubuntu@10.0.10.54` **depuis** le bastion. Deux commandes au lieu d'une ; ProxyJump reste le réflexe pro.

La preuve par curl : sortie oui, entrée non

Deux tests depuis l'instance privée, qui **prouvent** l'architecture :

```
TEST 1 – la sortie passe par la NAT GW :
ubuntu@privée:~$ curl -s https://checkip.amazonaws.com
15.237.xxx.xxx      <- c'est l'EIP de la NAT Gateway,
                    PAS une IP de l'instance (elle n'en a pas !)

TEST 2 – l'entrée directe est impossible :
vous@poste:~$ ssh ubuntu@10.0.10.54 # IP privée, injoignable
vous@poste:~$ ssh ubuntu@<EIP de la NAT> # la NAT ne relaie PAS l'entrant
-> timeout dans les deux cas. Normal. C'EST le but.
```

`checkip.amazonaws.com` renvoie l'adresse **vue depuis Internet** : pour une instance privée, c'est forcément celle de la NAT. Un échec ici = une route ou un placement à revoir.

Ce couple de tests sera votre **preuve finale** au mini-TP de fin de journée.



Démo 3.1 (temps 1) — Le VPC complet, à la console

~60 min, tous ensemble, chacun sur SON compte. Suivez le formateur écran par écran :

1. **VPC** → **Create VPC** (« **VPC only** ») — pas l'assistant : on assemble pièce par pièce.
2. **4 sous-réseaux** sur 2 AZ — et les 5 IP réservées, constatées en vrai.
3. **IGW** : créer, puis attacher. **Table publique** : la route `0.0.0.0/0` → **IGW** + auto-assign IP.
4. **NAT Gateway** 💰 (l'encadré coût sera dit à voix haute) + EIP + **table privée**.
5. **Deux SG** : bastion (22 depuis votre IP) et privé (22 depuis... le SG bastion).
6. **Bastion + instance privée** (Ubuntu via SSM, t3.micro, comme au J2), `ssh -J`, et la **preuve** : `curl checkip` → l'IP de la NAT.



Script : `demos/06-cl-aws1/demo-3-1-vpc-complet.md`



Notez chaque identifiant créé (vpc-, subnet-, igw-, nat-...) dans un fichier texte : vous comprendrez pourquoi au temps 2.

6. « Tout est API »

Ce que vous avez cliqué en 40 minutes se scripte en 4

`create-vpc.sh` : votre matinée en 8 sections

Chaque clic de ce matin était un appel d'API. Le script `code/06-cl-aws1/create-vpc.sh` rejoue **exactement** la même construction :

Section	Appels CLI	Le clic console correspondant
1. VPC	<code>create-vpc</code> , <code>modify-vpc-attribute</code>	Create VPC + réglages DNS
2. Sous-réseaux	<code>create-subnet</code> ×4, <code>modify-subnet-attribute</code>	4 créations + auto-assign IP
3. IGW	<code>create-internet-gateway</code> , <code>attach-...</code>	créer puis attacher
4. Table publique	<code>create-route-table</code> , <code>create-route</code> , <code>associate-...</code>	la route qui rend public
5. NAT GW 💰	<code>allocate-address</code> , <code>create-nat-gateway</code> , <code>wait</code>	EIP + NAT + attente <code>available</code>
6. Table privée	<code>create-route-table</code> , <code>create-route</code> , <code>associate-...</code>	<code>0.0.0.0/0</code> → NAT
7. SG	<code>create-security-group</code> , <code>authorize-...-ingress</code>	dont <code>--source-group</code> (SG → SG)
8. <code>vpc-ids.env</code>	—	vos notes manuscrites... automatisées

Chaque ID capturé (`--query ... --output text`, réflexe du J1) nourrit la commande suivante : le script encode **l'ordre des dépendances**.

teardown-vpc.sh : détruire, c'est construire à l'envers

On ne supprime pas un VPC « d'un coup » : chaque ressource refuse de mourir tant qu'une autre dépend d'elle (`DependencyViolation` — vous la rencontrerez, c'est un rite de passage).

Ordre du script — l'exact **miroir inverse** de la création :

```
instances EC2 -> NAT GW (+ wait) -> EIP -> associations + tables
de routage -> IGW (détacher PUIS supprimer) -> SG (private avant
bastion : il le référence !) -> sous-réseaux -> VPC
```

Et il **vérifie qu'il ne reste rien** : `describe-nat-gateways`, `describe-addresses`, `describe-vpcs` — trois sorties qui doivent être **vides**.

💰 Le teardown n'est pas une option : NAT GW oubliée = **~1,20 \$/jour** ; EIP orpheline (non associée) = facturée aussi. Le script relit `vpc-ids.env` : rien à retenir, rien à oublier.



Démo 3.1 (temps 2) — On détruit tout... et on rejoue en script

~30 min. La deuxième moitié de la démo, et le message le plus important du jour :

1. **Destruction à la console**, dans le bon ordre — vous verrez la `DependencyViolation` en direct quand on triche sur l'ordre.
2. `export PREFIX=abc` puis `./create-vpc.sh` : le VPC complet renaît en ~4 minutes, section par section commentée à l'écran.
3. Vérifications : les describe, puis re-preuve `curl checkip` (optionnelle).
4. `./teardown-vpc.sh` : tout disparaît, vérification « il ne reste rien » incluse.

« **Ce qui prend 40 minutes à la console prend 4 minutes en script — reproductible, versionnable, sans oublier. C'est votre métier.** »

📄 Script : `demos/06-cl-aws1/demo-3-1-vpc-complet.md` (temps 2) Les deux scripts sont dans `code/06-cl-aws1/` — vous les réutilisez au TP dans 5 minutes.



Mini-TP — Reconstruire le VPC en autonomie

90 min, en autonomie. NON NOTÉ — c'est votre répétition générale avant le TP2 (bloc 07), où ce réseau portera StockLine... et une note.

Mission : reconstruire **l'intégralité** du VPC pédagogique (mêmes CIDR que la démo) + bastion + instance privée. **Console ou script, au choix** — mais chaque étape a un point de contrôle `describe-...` vérifiable.

Preuve finale exigée (les deux) :

1. depuis l'instance privée, `curl -s https://checkip.amazonaws.com` → l'EIP de la NAT GW ;
2. le `ssh` direct vers l'instance privée depuis votre poste **échoue**.

📄 Sujet : `tp/06-cl-aws1/tp-mini-vpc.md`

💰 **TEARDOWN SCRIPTÉ OBLIGATOIRE** en fin de TP : `./teardown-vpc.sh` + vérification « il ne reste rien ». Personne ne quitte la salle avec une NAT Gateway vivante. Checklist d'auto-validation incluse dans le sujet.

Récap — les 10 points à retenir

1. Un VPC = un bloc CIDR **régional** ; un sous-réseau = **une seule AZ**.
2. AWS réserve **5 IP** par sous-réseau (un /24 → 251 utilisables).
3. Un sous-réseau est public à cause d'**une route** (`0.0.0.0/0` → **IGW**), pas d'un nom.
4. IGW : **créer puis attacher** — deux opérations (et l'inverse au teardown).
5. La NAT GW vit en sous-réseau **public**, exige une **EIP**, et 💰 **facture à l'heure**.
6. Table privée : `0.0.0.0/0` → **NAT GW** = sortie sans entrée.
7. SG = **stateful, allow only**, attaché à l'instance ; source possible = **un autre SG** (patron 3-tiers).
8. NACL = **stateless** (ports éphémères 1024-65535 au retour !), règles **numérotées** (la plus basse gagne), sait **deny** — sinon, laissez la défaut tranquille.
9. Bastion + `ssh -J` : la clé ne quitte jamais votre poste ; preuve = `curl checkip` → IP de la NAT.
10. **Tout est API** : 40 min de clics = 4 min de script. Et le **teardown du soir** est un rituel. 💰

Quiz de fin de journée

10 questions — réponses en débriefing avec le formateur

Quiz — questions 1 et 2

Question 1. Qu'est-ce qui rend un sous-réseau « public » chez AWS ?

- A. Son nom contient « public »
- B. Une case « public » cochée à sa création
- C. Sa table de routage contient une route `0.0.0.0/0` vers une Internet Gateway
- D. Le fait qu'il se trouve dans la première AZ de la région

Question 2. Combien d'adresses IP sont utilisables pour vos instances dans un sous-réseau AWS en `10.0.0.0/24` ?

- A. 256
- B. 254
- C. 251
- D. 250

Quiz — questions 3 et 4

Question 3. Où doit vivre une NAT Gateway, et de quoi a-t-elle besoin ?

- A. Dans un sous-réseau privé, avec une IP privée
- B. Dans un sous-réseau public, avec une Elastic IP
- C. Hors du VPC, comme l'IGW
- D. Dans chaque sous-réseau privé qu'elle doit servir

Question 4. Un SG autorise le 22/tcp entrant depuis votre IP, et ses règles **sortantes** ont toutes été supprimées. Que devient votre session SSH entrante ?

- A. Elle échoue : le retour est bloqué en sortie
- B. Elle fonctionne : le SG est stateful, le retour d'une connexion autorisée passe automatiquement
- C. Elle fonctionne, mais en lecture seule
- D. Elle échoue une fois sur deux

Quiz — questions 5 et 6

Question 5. Vous devez **interdire** toute connexion depuis l'adresse précise 203.0.113.66, sans rien changer d'autre. Quel mécanisme le permet ?

- A. Une règle « deny » dans le security group
- B. Une règle deny dans la NACL, avec un numéro plus bas que les règles allow
- C. Supprimer la route `local` du VPC
- D. Ce n'est pas possible sur AWS

Question 6. Une NACL personnalisée autorise en entrée le 443/tcp depuis Internet, et n'a **aucune règle sortante** (hormis le deny * final). Un client HTTPS se connecte. Que se passe-t-il ?

- A. La connexion fonctionne : l'entrée est autorisée
- B. Le SYN entre, mais la réponse est bloquée en sortie (port éphémère non autorisé) : timeout
- C. La connexion est refusée immédiatement avec une erreur claire
- D. La NACL ne s'applique pas au HTTPS

Quiz — questions 7 et 8

Question 7. Dans une règle entrante de security group, que signifie mettre **un autre SG** comme source ?

- A. Les deux SG fusionnent leurs règles
- B. Toute instance portant ce SG source est autorisée, quelle que soit son adresse IP
- C. Seule l'instance la plus ancienne du SG source est autorisée
- D. La règle ne s'applique qu'entre AZ différentes

Question 8. Un sous-réseau AWS peut-il s'étendre sur plusieurs zones de disponibilité ?

- A. Oui, c'est même recommandé pour la haute disponibilité
- B. Oui, mais uniquement pour les sous-réseaux publics
- C. Non : un sous-réseau vit dans une seule AZ ; la haute dispo = plusieurs sous-réseaux
- D. Non, sauf si le VPC est en `/16`

Quiz — questions 9 et 10

Question 9. Une NACL contient : règle **100** = deny 22/tcp depuis 203.0.113.0/24, règle **200** = allow 22/tcp depuis 0.0.0.0/0. Une connexion SSH arrive depuis 203.0.113.66. Résultat ?

- A. Autorisée : la règle 200 couvre tout le monde
- B. Bloquée : la règle 100 correspond en premier et l'évaluation s'arrête
- C. Autorisée : allow gagne toujours sur deny
- D. Cela dépend du security group

Question 10. Pourquoi le teardown de fin de journée est-il non négociable aujourd'hui, alors qu'il ne l'était « que » par principe les jours précédents ?

- A. AWS supprime les VPC inactifs de toute façon
- B. La NAT Gateway facture ~0,05 \$/h même inutilisée (et une EIP non associée est facturée aussi)
- C. Le quota de VPC est de un par compte
- D. Les sous-réseaux expirent après 24 h

À demain !

Regardez le chemin parcouru : le plan d'adressage dessiné sur papier au bloc réseau est devenu, aujourd'hui, un **vrai réseau** — cliqué le matin, **scripté le soir**, détruit proprement. Bastion, NAT, SG en référence croisée : vous avez construit l'ossature de toutes les architectures à venir.

Demain — Jour 4 : S3, le stockage infini

- Des **buckets** et des objets : le service le plus utilisé d'AWS — 11 « 9 » de durabilité.
- **Héberger le front de StockLine** : votre premier site servi directement depuis S3.
- Le premier duo **EC2 + S3 avec un rôle IAM** : une instance qui accède à un bucket **sans aucune clé stockée** sur la machine — la suite logique d'IMDSv2 vu au J2.
- Versioning, politiques de bucket, et les fuites de données célèbres qu'on ne reproduira pas.

D'ici là, une seule chose à vérifier : `./teardown-vpc.sh` est passé, et `aws ec2 describe-nat-gateways --profile formation --region eu-west-3 --filter "Name=state,Values=available"` ne renvoie **rien**. Bonne soirée — votre facture vous remercie. À demain !